2020 5th International Conference on Advanced
Robotics and Mechatronics (ICARM)

Model-Based Reinforcement Learning For Robot Control

Xiang Li, Weiwei Shang, and Shuang Cong

***Abstract*— Model-free deep reinforcement learning (MFRL) algorithms have achieved many impressive results. But they are generally stricken with high sample complexity, which puts forward a critical challenge for their application to real-world robots. Dynamic models are essential for robot control laws, but it is often hard to obtain accurate analytical dynamic models. Therefore a data-driven approach to learning models becomes significant for reinforcement learning to increase data efficiency. Model-based algorithms are effective methods to reduce sample complexity by learning the system dynamic model. However, in certain environments, it has been proven that learning an accurate system dynamic model is a formidable problem, and their asymptotic performance cannot achieve to the same level as model-free algorithms. In our work, we use an ensemble of deep neural networks to learn system dynamics and incorporate model uncertainty. Then in order to merge the high asymptotic performance of the advanced model-free methods, the deep deterministic policy gradient (DDPG) algorithm is adopted to optimize robot control policy. Furthermore, it has been implemented within ROS for controlling a Baxter robot in the simulation environment.**

I. INTRODUCTION

In recent years, MFRL algorithms have been proven to be able to learn a variety of robotic skills, including playing Atari games [1], mastering the game of Go [2] as well as learning complex locomotion skills [3] [4]. These algorithms are generally applicable, and can easily merge high expressive function approximators including deep neural networks. However, in order to achieve great performance, these algorithms are stricken with high sample complexity, generally requiring hundreds of millions of samples for a certain control task. Therefore their applications are mainly limited to simulated environments. Reducing sample complexity requires either obtaining more information from available data or having informative prior knowledge.

Models are one of the most crucial parts in robotics, for instance, the kinematic and dynamic models of the robot and external controllable objects. Classical robotics generally depends on analytical models which are derived from human interpretations of physics, such as the Newton Euler method and the Lagrangian method [5]. However, it is hard to obtain the accurate system analytical models, because of the complication of robot systems and their presence in human-inhabited, uncertain, and unstructured environments. Data-driven model learning can be an effective method to replace manual pre-programming, if the model can be learned directly from sensor data. By learning a model of the system dynamics, model-based reinforcement learning (MBRL) algorithms exploit the information from state observations, and can possibly be much more data-efficient than the model-free algorithm. Therefore, these methods can be applied to real-world robots [6] [7] [8]. The framework of our model-based algorithm is shown in Fig. 1. The dynamic model is a crucial part of MBRL [9] since the policy performance in the real environment relies on the prediction accuracy of the model. However, it has been proven that learning an accurate system dynamic model is a formidable problem in continuous high dimensional state space control tasks. And policy optimization tends to take advantage of areas where available data is insufficient to train the dynamic model, which may cause catastrophic failures, known as model-bias [6] [10].

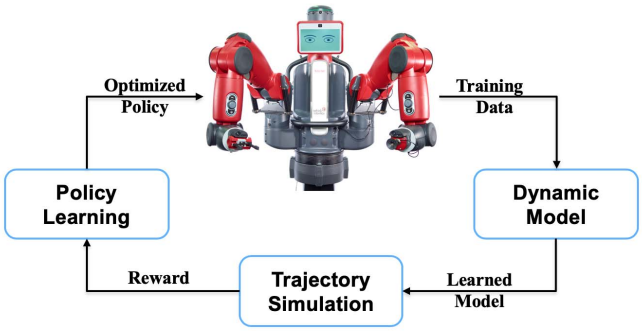


Fig. 1: The framework of our model-based reinforcement learning algorithm. The training dataset gathered from the robot's sensors in the real environment is used to train the dynamic model. The robot control policy is optimized by model-free algorithm. And the learned dynamic model is used for trajectory simulation. After policy learning, the policy will be applied to the robot.

We design an ensemble of feed-forward neural networks to estimate system dynamic model uncertainty given the data gathered from robot's sensors in a real environment. We distinguish the neural networks by randomly initialing their weights and selecting a training dataset during model learning. Then, in order to merge the high asymptotic performance of the model-free algorithm, we use the DDPG algorithm to optimize robot control policy. Each step state is the mean value of the ensemble predictions. Using the model-free algorithm, the policy learns to become robust against sensory noise and various potential situations in real environment. Finally, we empirically demonstrate on Baxter robot control task in the Gazebo simulation environment.

Our research work was supported by the NSFC with Grant 51675501 and 51275500, the YIPA-CAS with Grant 2012321, and the SKLRS with Grant SKLRS-2018-KF-07.

Xiang Li, Weiwei Shang, and Shuang Cong are with the Department of Automation, University of Science and Technology of China, Jinzhai Road 96, Hefei 230027, P. R. China li12@mail.ustc.edu.cn, wwshang@ustc.edu.cn, scong@ustc.edu.cn.

978-1-7281-6479-3/20/$31.00 ©2020 IEEE

Authorized licensed use limited to: St Petersburg Natl Uni of Info Tech Mech & Optics. Downloaded on September 29,2024 at 08:25:55 UTC from IEEE Xplore. Restrictions apply.

II. RELATED WORK

In terms of sample complexity, model-based methods generally outperform model-free methods [11] and have been successfully applied to robotic systems in the real world, such as remote-controlled cars [12], pendulums [6] and soft robotic manipulators [8]. But the most effective model-based methods generally use comparatively simple function approximators, for instance, linear models [12] [13], Gaussian processes(GPs) [6] [7] and mixtures of Gaussians [14]. These models are data-efficient and optimize policy efficiently through optimal control theory. However, due to limited expressiveness, they cannot adjust well to high-dimensional state and action space or complicated nonlinear dynamics.

Recent improvements in parametric function approximators, such as Neural networks, which are easily able to adjust to complex high dimensional state and action space, and have constant-time inference and tractable training. Therefore, neural networks for model learning provide the possibility to scale to high-dimensional tasks, including non-smooth dynamics that are often present in robotics [15] [16]. The major challenges in building a model that performs well in low and high data regimes: in the early stage of training, highly expressive function approximators are liable to overfit; in the later stage of training, simple function approximators might underfit for complex dynamics. Several works prevent policies from exploiting model-bias through learning an ensemble of deterministic models [17] [15] [16] or probabilistic models [18].

Naturally, in order to acquire the high asymptotic performance and the low sample efficiency, it is desirable to merge model-free and model-based methods. Attempts are roughly categorized into three main methods [10]. First is Dyna-style Algorithms [19] [17] [16]. Training iterates between two steps: training data is collected from the real environment and used to train the dynamic model. The policy is optimized with virtual trajectories simulated by the learned dynamic model. Second is Policy Search with backpropagation-through-time methods [6] [7]. The learned dynamic model is used to provide simulated data, and policy search takes advantage of the model derivatives. Accordingly, such algorithms are capable to calculate analytic gradient of the reinforcement learning objective and then optimize control policy. Third is Shooting Algorithms [15] [18]. These methods use model predictive control to deal with non-linear dynamics and non-convex reward functions

III. ALGORITHM

A. Model-based Reinforcement Learning

We detail the model-based algorithm framework and the notation used. According to the Markov decision process formulation, we formulate the task as the finite-horizon discrete-time Markov decision process (MDP) that is denoted by the tuple $(\mathcal{S}, \mathcal{A}, f, r, \rho_0, \gamma, H)$ where $\mathcal{S}$ is state space, $\mathcal{A}$ is action space. In this work, we mainly concentrate on the continuous state and action space. $f : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ defines the transition dynamic function, $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ defines the reward function, $p_0 : \mathcal{S} \rightarrow \mathbb{R}_+$ denotes the initial state distribution, H is the control horizon of problem and $\gamma \in [0, 1]$ denotes the discount factor. We define the expected return $\eta(\theta) = \mathbb{E}_\tau \left[\sum_{t=0}^{H-1} r\left(\boldsymbol{s}_t, \boldsymbol{a}_t\right)\right]$, where $\tau := (\boldsymbol{s}_0, \boldsymbol{a}_0, \ldots, \boldsymbol{s}_{H-1}, \boldsymbol{a}_{H-1}, \boldsymbol{s}_H)$ denotes the whole trajectory and $\boldsymbol{s}_{t+1} = f\left(\boldsymbol{s}_t, \boldsymbol{a}_t\right)$. The objective of reinforcement learning is to optimize the policy $\pi_{\boldsymbol{\theta}}(\boldsymbol{a}|\boldsymbol{s})$ to maximize the expected return $\eta(\boldsymbol{\theta})$.

Model-free methods cannot explicitly model state transitions, while model-based algorithms model the transition dynamic function from the observed transitions, also known as dynamic model. After gathering trajectories from the real environment, the robot state transitions are stored in the dataset $\mathcal{D} = \{(\boldsymbol{s}_t, \boldsymbol{a}_t, \boldsymbol{s}_{t+1})\}$. Then the dataset is used to train a dynamic model $\hat{f}_{\boldsymbol{\phi}}$ with a parametric function approximator. Once a dynamic model $\hat{f}_{\boldsymbol{\phi}}$ is learned, we use it to predict the next state resulting from applying a given policy. Alternatively, this dynamic model can be used as an internal simulator to generate more training dataset that is then used for policy learning.

B. Model Learning

In this work, we consider the system dynamic model as the deterministic function of the current action $\boldsymbol{a}_t$ and the state $\boldsymbol{s}_t$. Therefore, we use an ensemble of deep neural networks to approximate real system dynamics. We design the identical network architecture: 2-hidden layer NN with ReLU nonlinearity, including 256 hidden units. We randomly initial their weights and sample training subset from the gathered real environment experience so as to distinguish the models.

The standard practice is adopted to train the dynamic models to predict the change of state, i.e. $\Delta \boldsymbol{s} = \boldsymbol{s}_{t+1} - \boldsymbol{s}_t$ instead of the next state $\boldsymbol{s}_{t+1}$. When the change is small, this method prevents the dynamic model from memorizing the input state. [6] [17] [15] [16]. Let $\hat{f}_\phi$ denote the prediction of the next state, which is the sum of the output of neural networks and the input state. The objective for training the model $\hat{f}_{\boldsymbol{\phi}_k}$ of the ensemble is to optimize the networks' parameter $\boldsymbol{\phi}_k$ to minimize the ℓ_2 state prediction loss:

$$\min_{\boldsymbol{\phi}_k} \frac{1}{|\mathcal{D}_k|} \sum_{(\boldsymbol{s}_t, \boldsymbol{a}_t, \boldsymbol{s}_{t+1}) \in \mathcal{D}_k} \left\| \boldsymbol{s}_{t+1} - \hat{f}_{\boldsymbol{\phi}_k}\left(\boldsymbol{s}_t, \boldsymbol{a}_t\right) \right\|_2^2 \quad (1)$$

where $\mathcal{D}_k$ is a randomly sampled subset from the training dataset $\mathcal{D}$. The standard techniques to promote fast and avoid overfitting learning are followed: 1) early stop, 2) normalize the outputs and inputs, and 3) normalize weight of the neural network [20].

After learning an ensemble of NN models $\{\hat{f}_{\boldsymbol{\phi}_1}, \hat{f}_{\boldsymbol{\phi}_2}, \ldots, \hat{f}_{\boldsymbol{\phi}_K}\}$ for a particular environment, we define $\hat{f}_{\boldsymbol{\phi}} = \frac{1}{K} \sum_{k=1}^{K} \hat{f}_{\boldsymbol{\phi}_k}$ as the dynamic model, where K denotes the size of the ensemble.

C. Policy Optimization

The main goal of reinforcement learning is to maximize the expected return $\eta(\boldsymbol{\theta})$ given an MDP, $\mathcal{M}$. During the

Authorized licensed use limited to: St Petersburg Natl Uni of Info Tech Mech & Optics. Downloaded on September 29,2024 at 08:25:55 UTC from IEEE Xplore. Restrictions apply.

policy optimization, model-based algorithms focus on the approximate MDP, $\hat{\mathcal{M}}$, with the learned system dynamic model $\hat{f}_{\boldsymbol{\phi}}$. The policy is optimized based on $\hat{\mathcal{M}}$ MDP. Therefore, the objective is

$$\hat{\eta}(\boldsymbol{\theta};\boldsymbol{\phi}) := \mathbb{E}_{\hat{\tau}}\left[\sum_{t=0}^{H-1} r\left(\hat{\boldsymbol{s}}_t, \boldsymbol{a}_t\right)\right] \tag{2}$$

where $\hat{\tau} = (\boldsymbol{s}_0, \boldsymbol{a}_0, \hat{\boldsymbol{s}}_1, \boldsymbol{a}_1 \ldots, \hat{\boldsymbol{s}}_{H-1}, \boldsymbol{a}_{H-1})$, $\boldsymbol{s}_0 \sim \rho_0(\cdot)$, $\boldsymbol{a}_t \sim \pi_\theta\left(\cdot|\hat{\boldsymbol{s}}_t\right)$, and $\hat{\boldsymbol{s}}_{t+1} = \hat{f}_\phi(\hat{\boldsymbol{s}}_t, \boldsymbol{a}_t)$.

DDPG algorithm [21] is a model-free reinforcement learning algorithm in the continuous action spaces combining the actor-critic approach with experience Deep Q Network (DQN). This method uses two deep neural networks: the actor is a deterministic policy π and the critic is an approximator of Q function $Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$, which are both parameterized through parameters vector $\boldsymbol{\theta}^\pi$ and $\boldsymbol{\theta}^Q$. The actor network maps input states to actions and tries to maximize $Q(\boldsymbol{s}_t, \boldsymbol{a}_t)$. DDPG uses the critic neural network to estimate Q function through minimizing the loss:

$$L_{\text{critic}} = \left(Q\left(\boldsymbol{s}_t, \boldsymbol{a}_t\right) - y_t\right)^2, \tag{3}$$

where y_t is the one-step target.

In order to constrain the target values to change slowly and improve the stability of learning, DDPG creates a copy of critic and actor networks as target Q network $Q^*(\boldsymbol{s}_t, \boldsymbol{a}_t|\boldsymbol{\theta}^{Q^*})$ and target policy $\pi^*(\boldsymbol{s}_t|\boldsymbol{\theta}^{\pi^*})$. The weights of target networks are updated slowly by tracking the learned networks: $\boldsymbol{\theta}^* = \tau\boldsymbol{\theta} + (1-\tau)\boldsymbol{\theta}^*$, $\tau \ll 1$. The target value y_t is computed by target policy π^* and target Q network Q^* as

$$y_t = r_t + Q^*\left(\boldsymbol{s}_{t+1}, \pi^*\left(\boldsymbol{s}_{t+1}\right)\right) \tag{4}$$

In order to remove samples' temporal correlations, the transition tuples $(\boldsymbol{s}_t, \boldsymbol{a}_t, r_t, \boldsymbol{s}_{t+1})$ are stored in a replay buffer and then the actor and critic networks are trained by sampling mini-batches uniformly from the replay buffer.

After learning the dynamic model, we use DDPG algorithm to optimize policy, until the control policy performs well in the dynamic model and in the real environment. The complete procedure for our model-based reinforcement learning algorithm is described in Algorithm 1.

Algorithm 1 Our model-based reinforcement learning

Initialize an ensemble of models $\{\hat{f}_{\phi_1}, \hat{f}_{\phi_2}, \ldots, \hat{f}_{\phi_K}\}$.
Define the dynamic model $\hat{f}_\phi = \frac{1}{K}\sum_{k=1}^{K}\hat{f}_{\phi_k}$.
Initialize a policy π_θ.
repeat
  Gather dataset $\mathcal{D}$ of designed trajectories.
  for $k = 1$ to K **do**
    randomly sample training subset $\mathcal{D}_k$, and train the model $\hat{f}_{\phi_k}$.
  end for
  Collect fictitious samples form $\hat{f}_\phi$ using π_θ.
  Optimize policy using DDPG.
  Estimate performance $\hat{\eta}(\theta, \phi)$.
  Estimate performance in the real environment.
until This control policy performs well in the real environment.

IV. EXPERIMENTS

Baxter robot is a collaborative robot built by Rethink Robotics with two 7 degrees of freedom (DOF) manipulators. Each manipulator contains 7 joints with an interchangeable gripper attached to the end-effector, illustrated in Fig. 2. The DH parameters are shown in Table I. The Baxter robot has position, velocity, and torque three different types of control modes. We use a simulated Baxter robot and mainly focus on the right arm. The robot states contain joint positions and joint velocities, therefore the dimension of state space is 14. The robot used in the experiments is operated in torque control mode. Fig. 3 illustrates the Baxter robot environment in Gazebo.

The training dataset contains 10 designed trajectories in joint space and 20000 samples are obtained at 20Hz. The

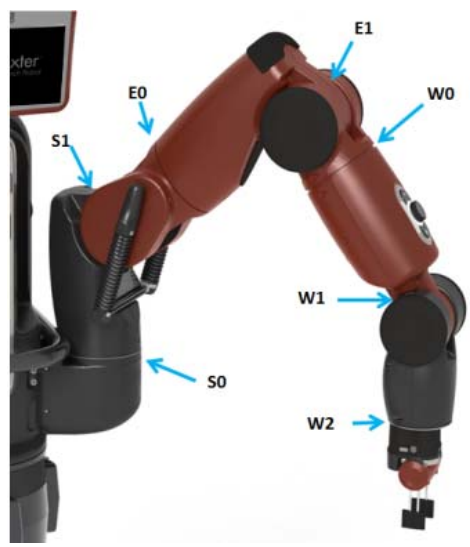


Fig. 2: The 7 joints of the Baxter robot arm. Joints S0, S1 comprise the shoulder, E0, E1 the elbow, and W0, W1, W2 the wrist.

TABLE I: DH parameters of the Baxter arm

Link	θ	d(m)	a(m)	α(rad)
1	θ_1	0.2703	0.069	-$\frac{\pi}{2}$
2	θ_2	0	0	$\frac{\pi}{2}$
3	θ_3	0.3644	0.069	-$\frac{\pi}{2}$
4	θ_4	0	0	$\frac{\pi}{2}$
5	θ_5	0.3743	0.01	-$\frac{\pi}{2}$
6	θ_6	0	0	$\frac{\pi}{2}$
7	θ_7	0.2295	0	0

Authorized licensed use limited to: St Petersburg Natl Uni of Info Tech Mech & Optics. Downloaded on September 29,2024 at 08:25:55 UTC from IEEE Xplore. Restrictions apply.

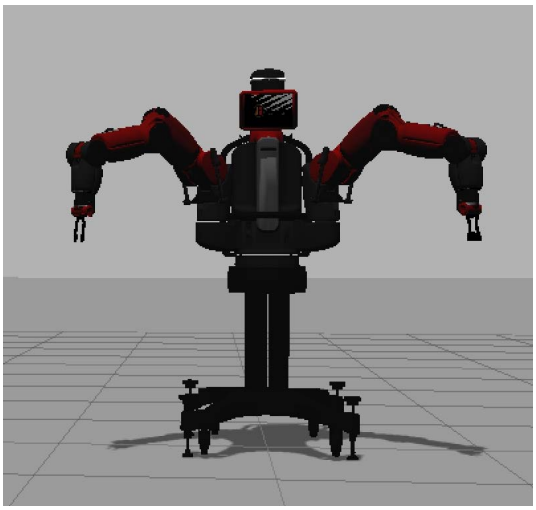


Fig. 3: Baxter robot simulation environment in Gazebo.

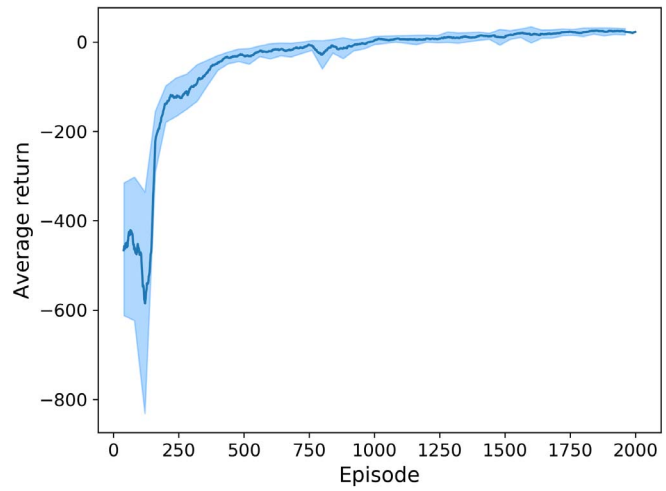


Fig. 4: The learning curve of policy learning. The vertical axis denotes the average return. The horizontal axis represents the number of training episodes.

dataset consists of joint positions, velocities, and applied torque values for all joints at each time step. The designed trajectories must ensure that joint positions and velocities are within a specified safe range. The trajectories are generated joint-wise, i.e., for the i-th joint the trajectory is defined by

$$q_d^i(t) = q_0^i + \sum_k \left[A_1^{ik} \sin\left(2\pi\omega_1^{ik} t\right) + A_2^{ik} \cos\left(2\pi\omega_2^{ik} t\right)\right] \tag{5}$$

where q_0^i is the initial joint position, A^{ik} the amplitude, ω^{ik} the frequency, and k the number of sine and cosine components. The amplitudes and frequencies are randomly chosen to minimize repetition in order to explore joint space as much as possible and to prevent the model from being able to learn a Fourier transform rather than the true dynamic model. In our experiments, the amplitudes and frequencies are sampled uniformly from $(-0.5, 0.5)$ and $(0.1, 0.3)$.

During policy learning, the 7-DOF manipulator has to reach arbitrary joint positions. The reward function consists of the energy/control costs and the distance of the joint positions to the goal:

$$\begin{aligned} r(\boldsymbol{s}, \boldsymbol{a}) = &- \|\boldsymbol{s}_{\text{pos}} - \boldsymbol{s}_{\text{goal}}\|_2^2 - \log \|\boldsymbol{s}_{\text{pos}} - \boldsymbol{s}_{\text{goal}}\|_2 \\ &- 0.05 * \|\boldsymbol{a}\|_2^2 \end{aligned} \tag{6}$$

The policy input contains the robot states and the joint position error $\boldsymbol{e}_t = \boldsymbol{s}_{\text{goal}} - \boldsymbol{s}_{\text{pos}}$ and outputs robot joint torque commands and hence the control policy is represented using the mapping: $(\boldsymbol{s}_t, \boldsymbol{e}_t) \rightarrow \boldsymbol{a}_t$. The control horizon is 60, which means 3 seconds in the real environment. The policy learning result is shown in Fig. 4.

Although we learn to predict one-step state well, when the dynamic model is used to predict multi-step step, one-step model-bias will accumulate. If the sample frequency is too high, the task control horizon will be too large. The policy optimized from the learned dynamic model will not perform well in the real environment. Therefore, we choose the sample frequency as 20Hz. In the actual control phase, as the control frequency increases, the performance of the control policy will become better. The tracking error with the changing control frequency for the same trajectory is shown in Table II. Finally, we choose the control frequency as 200Hz in after experiments.

TABLE II: Control policy performance with changing control frequency

Frequency[Hz]	Tracking Error[$10^{-2} rad$]
20	3.54±5.04
50	1.50±2.11
100	0.96±1.36
200	0.92±1.28

In order to confirm the ability of the learned control policy, we design a trajectory tracking task to test the performance of the policy in joint and operational space.

In joint space, 10 joint position trajectories are selected for the i-th joint as $q_d^i(t) = q_0^i + A^i \sin\left(2\pi\omega^i t\right)$, where the frequency and amplitude are chosen randomly. The test trajectories are distinct from the training trajectories. Table III summarizes the performance of all trajectory tracking tasks over all joints and an example tracking result of one joint is shown in Fig. 5. The tracking errors are around $10^{-2} rads$.

TABLE III: Trajectory tracking performance over all joints in joint state.

Joint Name	Tracking Error[$10^{-2} rads$]
S0	1.17±1.90
S1	2.55±3.21
E0	0.93±1.42
E1	1.54±2.02
W0	0.88±1.23
W1	0.86±1.39
W2	0.95±1.21

Authorized licensed use limited to: St Petersburg Natl Uni of Info Tech Mech & Optics. Downloaded on September 29,2024 at 08:25:55 UTC from IEEE Xplore. Restrictions apply.

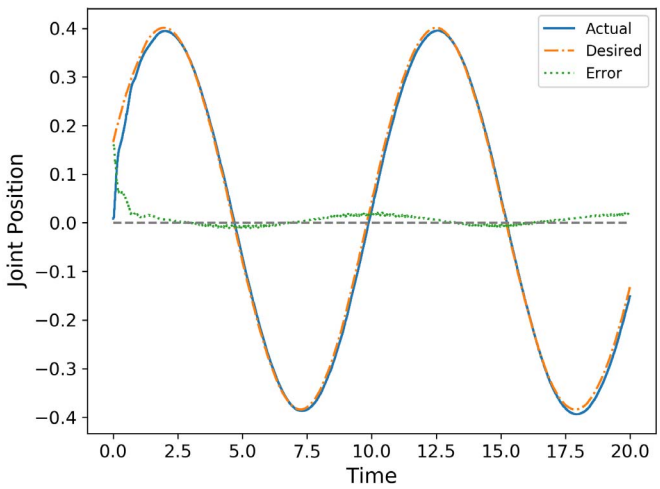
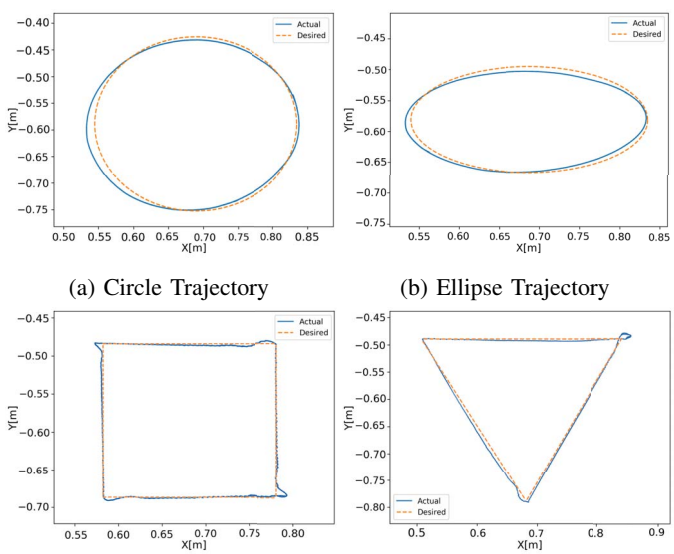


Fig. 5: Trajectory tracking task result in joint space. Orange, dash-dot line: desired trajectory; blue, solid line: actual trajectory controlled by the learned policy; green, dotted line: error between the actual and desired position of the joint.



(a) Circle Trajectory (b) Ellipse Trajectory

(c) Rectangle Trajectory (d) Triangle Trajectory

Fig. 6: Trajectory tracking results in X-Y plane in operational space.

TABLE IV: Mean Absolute Error for different shape trajectories in operational space.

Shape	MAE[m]
Circle	0.011
Ellipse	0.013
Rectangle	0.012
Triangle	0.009

Then in operational space, the performance of the policy has been evaluated on different shape trajectories (Circle, Ellipse, Rectangle, and Triangle) which are generated by MoveIt!. The performance is measured in terms of Mean Absolute Error (MAE) which is presented in Table IV and illustrations are presented in Fig. 6. The error is the distance between the actual and the desired position of the end-effector. The learned control policy shows high performance both in joint space and in operational space.

V. CONCLUSIONS

We present a robust model-based reinforcement learning algorithm that is capable to learn a deep neural network control policy for Baxter robot arm. We learn an ensemble of deep neural network models to effectively reduce model-bias. Combined with the high asymptotic of the model-free algorithm, the control policy is learned by DDPG with the learned models and perform well on the Baxter robot manipulator. In the simulation environment, the dynamic model only required a sampling period of 1000 s. Due to the model-free algorithm, the control policy can adapt to changes in control frequency and sensory noise. This method is a pure data-driven and data-efficient algorithm to directly learning control policy without the need of a manually analytical model. Our work justifies estimating uncertainty in the dynamic model explicitly and using a dynamic model for model-free sample complexity reduction. In future work, we intend to apply this model-based reinforcement learning algorithm to the real Baxter robot.

ACKNOWLEDGMENT

We would like to thank the ISLC-USTC for their hardware and software services.

REFERENCES

[1] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. A. Riedmiller, "Playing atari with deep reinforcement learning," *arXiv*, vol. abs/1312.5602, Dec. 2013.

[2] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. V. Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, *et al.*, "Mastering the game of go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, Jan. 2016.

[3] S. Levine, C. Finn, T. Darrell, and P. Abbeel, "End-to-end training of deep visuomotor policies," *Journal of Machine Learning Research*, vol. 17, pp. 1–40, Apr. 2015.

[4] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel, "Trust region policy optimization," *Computer Science*, pp. 1889–1897, Feb. 2015.

[5] D. Nguyen-Tuong and J. Peters, "Model learning for robot control: a survey," *Cognitive Processing*, vol. 12, no. 4, pp. 319–340, Apr. 2011.

[6] M. P. Deisenroth and C. E. Rasmussen, "Pilco: a model-based and data-efficient approach to policy search," in *Proceedings of the 28th International Conference on International Conference on Machine Learning, ICML*, Madison, WI, USA, Sep. 2011, pp. 465–472.

[7] M. Deisenroth, C. Rasmussen, and D. Fox, "Learning to control a low-cost manipulator using data-efficient reinforcement learning," in *Robotics: Science and Systems VII*, vol. 7, Los Angeles, CA, USA, Jun. 2011, pp. 57–64.

[8] T. G. Thuruthel, E. Falotico, F. Renda, C. Laschi, and S. Member, "Model-based reinforcement learning for closed-Loop dynamic control of soft robotic manipulators," *IEEE Transactions on Robotics*, vol. 35, no. 1, pp. 127–134, Feb. 2019.

Authorized licensed use limited to: St Petersburg Natl Uni of Info Tech Mech & Optics. Downloaded on September 29,2024 at 08:25:55 UTC from IEEE Xplore. Restrictions apply.

[9] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: a survey," *The International Journal of Robotics Research*, vol. 32, no. 11, pp. 1238–1274, Sep. 2013.

[10] T. Wang, X. Bao, I. Clavera, J. Hoang, Y. Wen, E. Langlois, S. Zhang, G. Zhang, P. Abbeel, and J. Ba, "Benchmarking model-based reinforcement learning," *arXiv*, pp. 1–25, 2019.

[11] M. P. Deisenroth, "A survey on policy search for robotics," *Foundations and Trends in Robotics*, vol. 2, no. 1-2, pp. 1–142, Aug. 2011.

[12] P. Abbeel, M. Quigley, and A. Y. Ng, "Using inaccurate models in reinforcement learning," in *Proceedings of the 23rd International Conference on Machine Learning, ICML*, New York, NY, USA, Jun. 2006, pp. 1–8.

[13] S. Levine and P. Abbeel, "Learning neural network policies with guided policy search under unknown dynamics," *Advances in Neural Information Processing Systems*, vol. 2, pp. 1071–1079, Jan. 2014.

[14] S. M. Khansari-Zadeh and A. Billard, "Learning stable nonlinear dynamical systems with gaussian mixture models," *IEEE Transactions on Robotics*, vol. 27, no. 5, pp. 943–957, Oct. 2011.

[15] A. Nagabandi, G. Kahn, R. S. Fearing, and S. Levine, "Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning," in *Proceedings - IEEE International Conference on Robotics and Automation*, Kuala Lumpur Convention Centre, Kuala Lumpur, Malaysia, Aug. 2018, pp. 7579–7586.

[16] I. Clavera, J. Rothfuss, J. Schulman, Y. Fujita, T. Asfour, and P. Abbeel, "Model-based reinforcement learning via meta-policy optimization," *arxiv*, vol. abs/1809.05214, 2018.

[17] T. Kurutach, I. Clavera, Y. Duan, A. Tamar, and P. Abbeel, "Model-ensemble trust-region policy optimization," in *6th International Conference on Learning Representations, ICLR - Conference Track Proceedings*, vol. 1, Vancouver, Canada, Feb. 2018, pp. 1–15.

[18] K. Chua, R. Calandra, R. McAllister, and S. Levine, "Deep reinforcement learning in a handful of trials using probabilistic dynamics models," *Advances in Neural Information Processing Systems*, pp. 4754–4765, May. 2018.

[19] R. S. Sutton, "Dyna, an integrated architecture for learning, planning, and reacting," *Intelligencesigart Bulletin*, vol. 2, no. 4, pp. 160–163, Apr. 1991.

[20] T. Salimans and D. P. Kingma, "Weight normalization: a simple reparameterization to accelerate training of deep neural networks," in *Advances in Neural Information Processing Systems*, Feb. 2016.

[21] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *Computer Science*, Sep. 2015.

[22] A. S. Polydoros and L. Nalpantidis, "Survey of model-based reinforcement learning: applications on robotics," *Journal of Intelligent and Robotic Systems: Theory and Applications*, vol. 86, no. 2, pp. 153–173, Jan. 2017.

[23] V. Feinberg, A. Wan, I. Stoica, M. I. Jordan, J. E. Gonzalez, and S. Levine, "Model-based value estimation for efficient model-free reinforcement learning," *arxiv*, vol. abs/1803.00101, Feb. 2018.

[24] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," *Advances in Neural Information Processing Systems*, vol. 2017-Decem, pp. 6403–6414, Dec. 2017.

Authorized licensed use limited to: St Petersburg Natl Uni of Info Tech Mech & Optics. Downloaded on September 29,2024 at 08:25:55 UTC from IEEE Xplore. Restrictions apply.